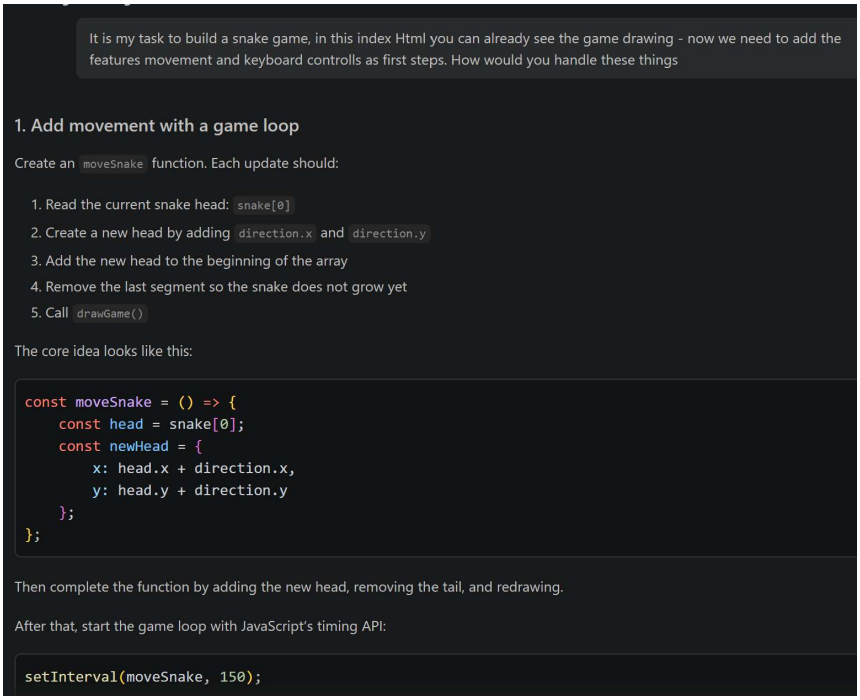
AI Assignment Report – Janick Hoffmann

1. Setup and Agent Configuration

Since I already set up GitHub Copilot for other university projects back home, I did not need to go through the student verification process. Configuring a custom AI agent using was a new experience for me. Because I found the starter agent to be well-designed, I kept it almost unchanged: constraining the tools strictly to read, search, and web capabilities, enforcing step-by-step guidance, and routinely prompting for git commits. The only real modification I made was tuning the tone of the conversation to make it more direct and solution oriented, since I wanted more efficient communication.

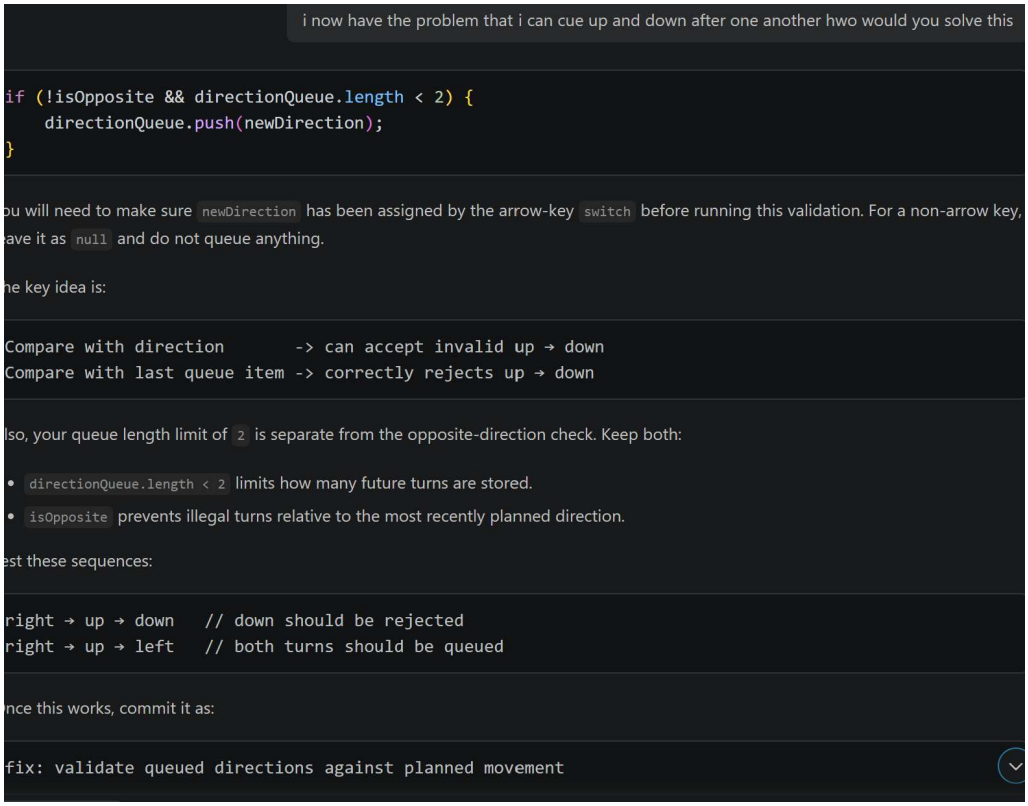
2. Development Process

At the start of the project, I asked the agent how to handle the basic movement and controls. It suggested creating a moveSnake() function with an array-based approach and setting up a game loop using setInterval. I followed this guidance, but as the project progressed, I added more logic to moveSnake(), like handling collision detection and eating food and growing.



Later on, I hit a bug with controls. To make moving the snake feel smoother, I implemented a queue that could store up to three key presses. However, this created a issue where you could make illegal direction changes (like turning directly into yourself). This happened because new inputs were being checked against the current direction instead of the last queued direction. With the help of the agent, I implemented an

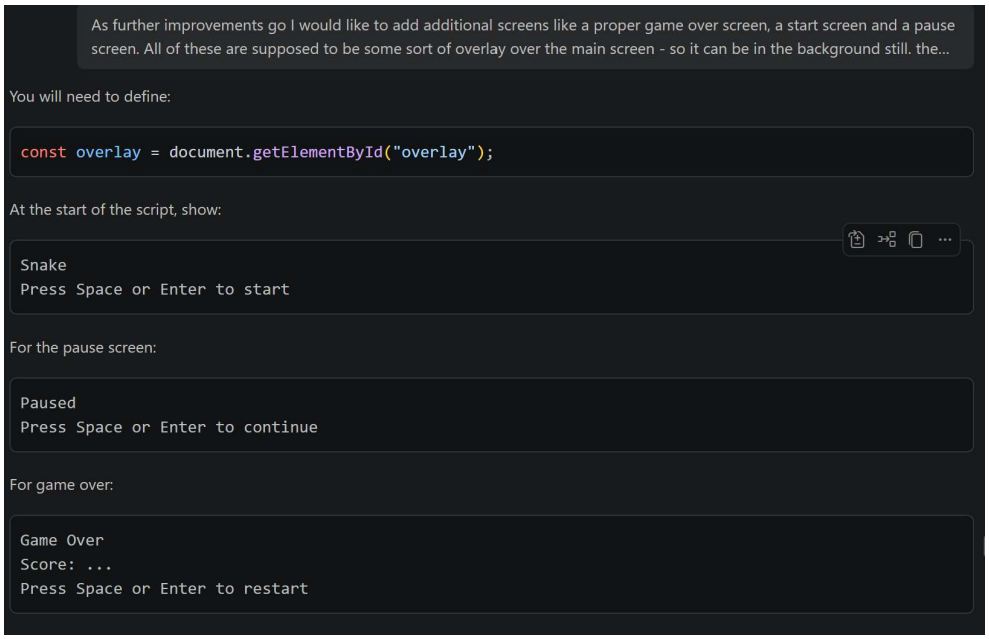
isOpposite check against the queued direction to prevent these illegal moves, which completely solved the problem.



When working on additional features, I wanted to speed up the snake as the score increased. I suggested using either a linear or an exponential function, and the agent helped me set up a proper exponential function for the move intervals. I followed the logic it provided, but I tuned the actual values myself to make the speed progression feel more to my liking.



Finally, I wanted to add a start screen, pause screen, and game over screen. The agent suggested building a UI overlay over the main game canvas, which I used. However, I didn't like its suggestions regarding the layout and look of the screens. Even though I wanted to keep them simple, I still wanted them to look a bit different.



3. Reflection

Looking back, changing the agent's tone to be more productive was definitely a useful tweak. If I were to do this again, I would also configure the agent to give shorter answers

with fewer direct code snippets, since I still ended up using a lot of the code suggestions directly without writing them myself.

Regarding inline suggestions, I found them really useful for quick repetitive tasks like changing variable names, which was especially helpful when implementing game states and high score mechanics. However, for general code writing, inline suggestions didn't always work out well. I think this might happen because there's no actual prompt input behind them. For example, when I built the keypress queue, the inline code suggestions weren't giving me what I needed, so I had to evaluate and solve the problem with the help of the Chat agent instead.

Regarding incorrectness of the agent, I dont think there was anything really incorrect. However sometimes the output was quite long so I decided against reading all of it and was just skimming it focussing on the main concepts and codes provided.

This project relied heavily on key JavaScript concepts from the lectures, such as setting up event listeners for key presses, defining variables properly with let and const and basic DOM manipulation to control the overlay screens. When the agent came up with suggestions I didn't like—such as the basic UI layouts—I simply took the underlying logic and customized the rest on my own.

In terms of the AI tool stages from the lectures, using GitHub Copilot with a custom .agent.md file shifts it away from pure automated code generation toward a cooperative, guided agent. Because it forces you to think through steps and ask questions rather than just generating a finished game, the control stays in the hands of the developer. I really liked using this step by step approach because I feel like it is similar to the pre AI way of coding where the developer is strongly in control but now has the support of a Agent that helps with problem solving and quick code generation.